

Pushing the Quadratic Boundary when Computing a Closest Pair

Author

September 22, 2026

Contents

1	Introduction	2
1.1	Duplicate Points	2
1.2	Points on a Line ($d = 1$)	2
2	Methodology	3
2.1	Language Choice	3
2.2	Point Generation	3
3	Low Dimensions	4
3.1	Two-dimensional Points ($d = 2$)	4
3.1.1	Divide-and-conquer	4
3.1.2	Sweepline	4
3.1.3	Grid Decomposition	5
3.2	Beyond Two Dimensions	5
3.2.1	Grid Decomposition (Revisited)	5
3.2.2	k -dimensional Tree	5

Chapter 1

Introduction

Computing the distance between two points is a common geometrical task. A natural extension to that would be to find the *closest pair* of points, i.e., the pair that yields the smallest distance. Assuming a point in d -dimensional space, computing the distance between two points takes $\mathcal{O}(d)$ time. There are $\binom{n}{2}$ pairs to check, forcing a total time complexity of $\mathcal{O}(d \cdot n^2)$.

In general, for large d , it's not possible to beat quadratic time in the *worst case*, assuming the **Strong Exponential Time Hypothesis (SETH)**. Luckily for us, there are tricks to salvage the situation at low dimensions, and for practical applications like vector databases, people have ditched the intractable task of obtaining an exact solution, opting for approximate algorithms instead.

1.1 Duplicate Points

Depending on the circumstances in which these points were generated, it's quite possible that we end up with duplicate points. For example, consider the Birthday Paradox. While it seems strange that only 23 people are required to produce a coin flip chance of having a shared birthday (the equivalent of two overlapping points), that's just how the probability checks out.

One way to solve this problem would be to first sort the points and check adjacent items to see if they're equal. Another approach is to inject them into a *hash set* one by one, and prior to the insertion, if the set already contains the point, we know a duplicate exists. The extra logarithmic factor in the former might seem burdensome, but it might actually beat the latter in terms of real runtime, considering hash sets in many languages are often plagued by high constant factors. Not to mention sorting also happens to be the first step in many of the subsequent algorithms, so the duplicate check arrives almost free in a sense.

1.2 Points on a Line ($d = 1$)

At $d = 1$, the points are just an array a of scalar values, so one may simply sort a and compare adjacent elements, consuming $\mathcal{O}(n \log n)$ time. If the range of values ($k = \max(a) - \min(a)$) is small enough, *counting sort* may be exploited to solve the problem in $\mathcal{O}(n + k)$.

At this juncture, it's natural to question if it's possible to achieve $\mathcal{O}(n)$ irrespective of k , and as it turns out, there are randomised algorithms with linear expected time. These algorithms are rooted in the idea of *decomposition*, and they generalise to higher dimensions as well. We'll elaborate more on the details when we get there.

Chapter 2

Methodology

2.1 Language Choice

The algorithms are implemented and tested in C++ for several reasons.

- A low-level compiled language gives you realistic estimates on how fast these algorithms actually are when they aren't held back by overheads like an interpreter.
- You get precise control over memory layouts and whether things go on the stack or the heap. For the most part, data structures like `std::array` are preferred because they reduce pointer chasing and are more cache-friendly.
- Multithreading is easy to perform in C++, thanks to tools like `std::jthread`. Although theoretical time complexities are important, whether or not an algorithm is practical and useful in real life also depends on how well it may be *parallelised*.

2.2 Point Generation

A meaningful dataset should try and avoid degenerate cases like duplicate points. This necessarily forces a large-enough range to circumvent the Birthday Paradox. In addition, it should be easy to compute the distance between any two points, and compare distances accurately. In other words, let's avoid floating point operations. Not only is it easy to mess up comparisons, when adding floats together the ideal strategy is to sort them by size and add the smallest numbers first before merging with the large ones. And this adds extra overhead that detracts from the core task at hand.

So `int` it is, and because we're working with *Euclidean* distances, a trick to preserve precision and reduce operations is to avoid the square root altogether, and compare squared distances instead. To spawn a set of points, we used the *Mersenne Twister* pseudorandom number generator (`std::mt19937_64`) to achieve a uniform random distribution of points. Fixed seeds are also used to ensure fair benchmarking.

Chapter 3

Low Dimensions

3.1 Two-dimensional Points ($d = 2$)

Numerous works have explored the two-dimensional case, and there exist well-known $\mathcal{O}(n \log n)$ solutions. Perhaps the most famous one is through divide-and-conquer, so let's begin with that.

3.1.1 Divide-and-conquer

Let's divide the set of points into two (approximately equal) halves. If n is odd, the first set will just be of size $\lceil \frac{n}{2} \rceil$ and the second set will be of size $\lfloor \frac{n}{2} \rfloor$. Intuitively, we want those with smaller x in one set and those with larger x in the other. For convenience, we can sort the points beforehand, with the sorting key being the point vector itself, i.e., perform element-wise comparison (check who has the smaller x , else if they have equal x , compare by y instead).

In the base case of $n = 1$ we simply return infinity (there's no pair!), else let's assume we have solutions to both halves. Let's take the minimum answer from each half (we'll denote this distance δ). Our goal is to potentially connect points from the two sets to achieve a new minimum. Consider the vertical divider ($x = x_{\text{mid}}$) used to partition the halves. Our candidate points are going to lie in the interval $(x_{\text{mid}} - \delta, x_{\text{mid}} + \delta)$. For some point (x_0, y_0) in this set, we may check just those in the interval $y \in (y_0 - \delta, y_0 + \delta)$. In fact we may just check those points with $y \leq y_0$, avoiding duplicate work.

This "box" of width $2 \cdot \delta$ and height δ contains $\mathcal{O}(1)$ points. Consider the left and right squares composing this box. If we partitioned each square into a grid of four smaller sub-squares of width $\frac{\delta}{2}$, you may fit at most one point into each sub-square. Even if we had placed points as far apart as possible, the distance is still $\frac{\delta}{\sqrt{2}}$. If two points fitted in a sub-square, that'll imply we should have gotten a smaller answer from one of the sub-problems, producing a contradiction.

To check points in this box easily, we'd like the points to be sorted by the y coordinate instead. This can be done exactly like how you'd perform *Merge Sort*. Surprisingly, the nature of our divide-and-conquer approach is no different from that of Merge Sort, and our time complexity is hence $\mathcal{O}(n \log n)$.

3.1.2 Sweepline

One downside of divide-and-conquer is the runtime overhead of the recursive calls. An alternative that yields the same time complexity (and also happens to be a bit faster empirically) is a sweepline approach. We begin by sorting the points like in divide-and-conquer. Next, we'll iterate through the points, and maintain a set of earlier points. This set organises the points in terms of y values.

Given the current minimum distance δ and the current point (x_0, y_0) , we only need to check points in the set with $y \in (y_0 - \delta, y_0 + \delta)$. Using a similar box analogy, one may prove we check

a constant number of points, producing an overall complexity of $\mathcal{O}(n \log n)$. Implementation-wise, we use C++’s `std::set` that uses a balanced binary search tree (specifically a red-black tree) under the hood.

3.1.3 Grid Decomposition

As alluded to at the start, there are algorithms that offer linear expected time. The trick is to visualise the search space as a grid, and group points into squares on a grid. To figure out the nearest pair, there are two cases to handle. First, iterate all pairs within a grid square. Second, considering a point within a square, let’s check its distance against all other points in adjacent squares. If we pick a good grid width, we’ll only run through a constant number of pair checks per grid square, thus yielding $\mathcal{O}(n)$ time.

Intuitively, the grid width should be neither too big nor too small. A grid that’s too fine-grained might make it impossible to find a neighbouring point in the vicinity, and a large grid width just degenerates into the brute force search. In a perfect world, we’d take the distance between the closest pair as the grid width, but that brings us right back to the original problem. Eventually, people discovered that taking the minimum distance of a bunch of random points (i.e., sample n pairs) produces a grid width that works surprisingly well.

Implementation-wise, we use the **SplitMix64** hash function paired with a hash map. At the moment we settled with the standard library’s `std::unordered_map`. Despite the amortised $\mathcal{O}(1)$ lookup, its closed addressing scheme isn’t particularly cache-friendly.

3.2 Beyond Two Dimensions

While the two-dimensional algorithms have proven encouraging, both theoretically and empirically, the real challenge is to solve the problem at higher dimensions, and this is where tools like divide-and-conquer begin to fall apart as you can’t easily generalise them.

3.2.1 Grid Decomposition (Revisited)

One neat property of grid decomposition is that it extends naturally to arbitrary dimensions. A square morphs into a hypercube, and now we need to check adjacent hypercubes instead. There’s one big problem though; at dimension d , there are $\frac{3^d-1}{2}$ neighbours to check, and beyond a certain point, you’re better off just brute-forcing.

3.2.2 k -dimensional Tree

A closely related problem to finding the closest pair is to handle point queries where given a certain point, we ask for the nearest point to it. Answering the main problem then becomes calling point queries on every point, and if we can perform this point query fast enough, we’ll beat the brute force approach.

As fate would have it, decades ago people already invented a data structure called the k -dimensional tree. In short, each node of this *binary tree* contains a point. To construct the tree, let’s assume we’re at the root. To decide which point gets to be the root, we’ll take the *median* point based on the first axis (in two dimensions that’ll just be x). Now, all points with x less than the median go to the left subtree and the rest go to the right subtree. Recurse down to the next subtree and this time, we’ll partition based on the second axis (i.e., y). Rinse and repeat till the tree is fully constructed. Note that the maximum depth of this tree might *not* be d , it could exceed d and if so we just loop back to the first axis.

To resolve a query, let us conduct a depth-first search from the root. At each node in the tree, let’s use it to improve the current score. Next, we need to figure out which subtree is likely to give us a better answer, and the instinct is to use the current axis as a basis of comparison.

For example, if x of our query point is greater than that of the current node, let's recurse the right subtree. What's important to realise is that this is just a *heuristic*, it's not guaranteed that the nearest point is going to be in the right subtree!

Denoting the gap along this axis as $g = x_q - x_m$, where x_q is the x coordinate of the query point and x_m is the x coordinate of the node, and without loss of generality assume that $g \geq 0$ (the right subtree case), then if $g \geq c_{\text{best}}$, where c_{best} denotes the current minimum distance, there's no point exploring the left subtree since we'll need to pay the cost of g which already exceeds our best answer.

Empirically, this data structure works great on our points because of the underlying uniform random distribution. During tree construction, we're almost always able to attain two subtrees of approximately equal size. As a result, the binary tree is very balanced, and the build stage consumes $\mathcal{O}(n \log n)$ time. Another phenomenon at low dimensions is that this pruning heuristic cuts off useless subtrees really well, allowing us to enjoy point queries in roughly $\mathcal{O}(\log n)$ time, leading to a total time complexity of $\mathcal{O}(n \log n)$. Of course, things break down at higher dimensions because points become sparse, and the contribution of each dimension towards the overall distance is tinier. Eventually, we end up needing to search both subtrees. Fortunately, it just degrades to the brute force, unlike general grid decomposition that mandates searching all neighbouring hypercubes.